🟦 Day 40 – Wrapping Up System Design

Anti-Patterns + Null Object Pattern

> **Day 40 mainly covers two things:**
>
> 1. Common **Anti-Patterns** that make software design worse
> 2. **Null Object Design Pattern** as a solution to repeated null checks

🟩 PART 1 — Anti-Patterns

What is an Anti-Pattern?

An **Anti-Pattern** is a common way of designing/writing code that may look convenient initially but can create **problems later**.

Simple:

```
Quick / Bad Design
       ↓
Looks easy initially
       ↓
Future requirements increase
       ↓
Problems appear
       ↓
Maintenance becomes difficult
```

Definition

> **Anti-Pattern = A bad/repeated design practice that creates problems in the future.**

It is basically the opposite idea of a Design Pattern.

```
Design Pattern → Good reusable solution

Anti-Pattern   → Common bad practice
```

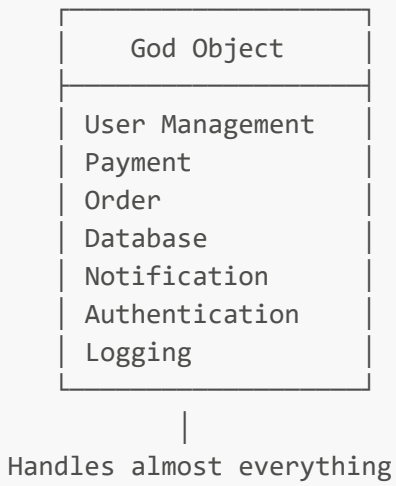
🟩 1.God Object

What is a God Object?

A **God Object** is a class/object that has **too many responsibilities** and handles a major portion of the application's functionality.

> Aise objects jisme bahut saare methods ho aur application ka major chunk God Object handle kar raha ho ya uske through handle ho raha ho.

Example:



❌ Problem

One class becomes responsible for too many things.

This generally creates:

- High coupling
- Difficult maintenance
- Difficult testing
- Large class
- Many reasons to change

Is an Orchestrator Class a God Object?

Your notes specifically raise this doubt:

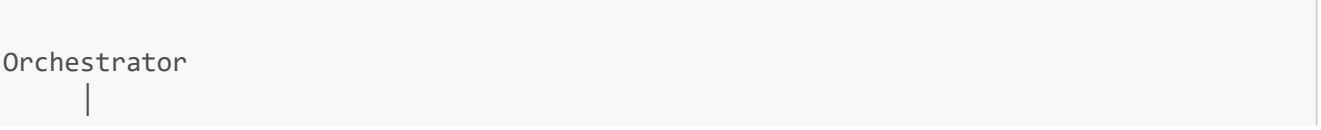
> **"Is our orchestrator class God Object?"**

The answer depends on **what the orchestrator actually does**.

There are two scenarios.

1.Delegating

Suppose:



```
      ├── UserManager
      ├── PaymentManager
      ├── OrderManager
      └── NotificationManager
```

Orchestrator simply delegates:

```
Orchestrator
      │
      ├── userManager.doTask()
      ├── paymentManager.doTask()
      └── orderManager.doTask()
```

If the orchestrator is **just delegating tasks to other classes**, then it is **not a God Object**.

Why?

Because actual work is being performed by specialized classes.

```
Orchestrator
     ↓ delegates
Specialized Classes
     ↓
Actual Work
```

2.Delegating With a Twist

This is the important case from your notes.

Suppose the object delegates the task, **but before delegating, it also performs significant work itself**.

```
Orchestrator
     │
     ├── Does some work
     │
     ↓
Delegates task
     │
     ↓
Other Class
```

Then it starts becoming a **God Object**.

Example

```
void placeOrder() {

    // lots of business logic here
    validateUser();
    calculatePrice();
    applyDiscount();
    checkInventory();

    // finally delegate
    orderManager.placeOrder();
}
```

The orchestrator is no longer just coordinating.

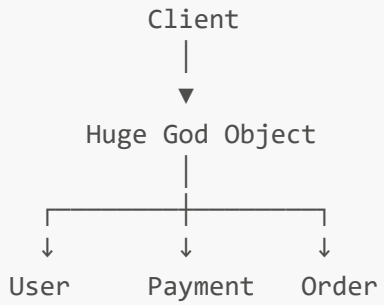
It is also doing business logic.

Solution to God Object

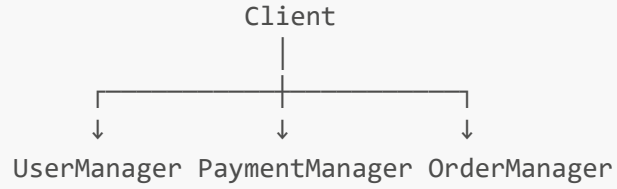
Your notes give two options.

Solution 1 — Create Multiple Objects

Instead of:



Create separate classes:



Now the client may need to know about these classes.

Solution 2 — Orchestrator with Delegation

Keep an orchestrator as a **single entry point**, but make sure it only coordinates/delegates.

```
Client
  |
  ▼
Orchestrator
  |
  ├──────► UserManager
  |
  ├──────► PaymentManager
  |
  └──────► OrderManager
```

Key Point

> **Orchestrator should coordinate, not contain all the business logic.**

Your notes also connect this with the idea of a **single entry point**, making it easier for the client.

Example mentioned:

```
API
 ↓
Controller
 ↓
What sort of layer...
```

The idea is that the client interacts through a controlled entry point instead of knowing every internal class.

🟩 2.Spaghetti Code

What is Spaghetti Code?

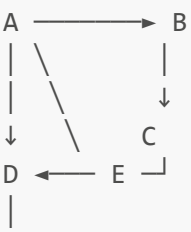
Spaghetti code means code that is **too complicated, tangled, and difficult to follow**.

> Making objects so complex that there is no clear entry point and no clear exit point.

And:

> Objects become too tightly coupled and there are no clear paths.

Diagram



```
└──→ F
 \      ↑
  └─────┘
```

Everything is connected with everything.

Problems with Spaghetti Code

1️⃣ Highly tightly coupled

```
Class A → B → C → D
       ↘ E ↗
```

Changing one class may affect many others.

2️⃣ Prone to errors

Because the flow is difficult to understand:

```
No clear flow
     ↓
Hard to debug
     ↓
More chances of mistakes
```

🟩 3.Hard-Coding Things

Your notes give this example:

```
void m1() {
    string s = "Hello";
}
```

Here `"Hello"` is directly written inside the code.

If tomorrow the value changes:

```
"Hello"
   ↓
"Hi"
```

we need to modify the source code.

Problem

```
Hard-coded value
      ↓
Requirement changes
      ↓
Source code modification
```

How to Avoid Hard-Coding?

Your notes suggest using a **constructor**.

Instead of:

```
void m1() {
    string s = "Hello";
}
```

we can provide the value from outside:

```
class Message {
    string message;

public:
    Message(string message) {
        this->message = message;
    }
};
```

Now:

```
Message m("Hello");
```

or:

```
Message m("Hi");
```

No need to change internal logic.

Idea

> **Values that may change should not unnecessarily be hard-coded inside business logic.**

🟩 4.Gold-Plating / Over-Engineering

Meaning

Gold-plating means trying to make the design **more complicated/perfect than actually required**.

Your notes mention:

- Trying to always achieve perfection in design
- Handling cases that will never arise
- Applying patterns unnecessarily
- Over-complicated code

Example

Requirement:

```
Simple calculator
+
-
*
/
```

But developer creates:

```
Calculator
 ├── AbstractCalculator
 ├── CalculatorFactory
 ├── CalculatorStrategy
 ├── CalculatorBuilder
 ├── CalculatorVisitor
 ├── CalculatorObserver
 └── CalculatorFacade
```

😂 This is unnecessary for a simple problem.

Rule

> **Don't use a Design Pattern just because you know it. Use it when the problem requires it.**

🟩 5.DRY — Don't Repeat Yourself

DRY means:

> **Don't Repeat Yourself.**

Your notes mention:

- Copy-pasting some code
- Too much repetition
- If change is needed, change has to be made everywhere the code is used

Bad Example

```
void calculateA() {
    cout << "Tax calculation";
    // same logic
}

void calculateB() {
    cout << "Tax calculation";
    // same logic
}

void calculateC() {
    cout << "Tax calculation";
    // same logic
}
```

Same logic is repeated.

If logic changes:

```
Change A
Change B
Change C
...
```

How to Avoid DRY Violation?

Your notes give two solutions:

1️⃣ Extract repeated code

Put repeated logic into a:

```
Method
   OR
Class
   OR
Separate Logic
```

Example:

```
void calculateTax() {
    // common logic
}
```

Then:

```
calculateTax();
```

can be reused.

2️⃣ Use Utility Class

Common reusable functionality can be placed in a utility class.

```
Utility
 ├── calculateTax()
 ├── validate()
 └── format()
```

🟩 6.Constructor Overloading / Telescoping Constructor

> Creating too many constructors.

Example:

```
User();
User(string name);
User(string name, int age);
User(string name, int age, string city);
User(string name, int age, string city, string email);
```

As parameters increase, constructors increase.

This is called **Telescoping Constructor Problem**.

Problem

```
Too many constructors
        ↓
Hard to understand
        ↓
Hard to maintain
```

Solution

Your notes give:

> **Builder Design Pattern**

Builder allows us to construct complex objects step-by-step.

```
UserBuilder
   │
   ├── setName()
   ├── setAge()
   ├── setCity()
   └── build()
```

🟩 7.Overuse of Getters / Setters

Your notes mention that providing getters/setters for **all private members by default** can be problematic.

❌ Common mistake

```cpp
class User {
private:
    string name;
    int age;
    double salary;

public:
    getName();
    setName();

    getAge();
    setAge();

    getSalary();
    setSalary();
};
```

Just because a variable is private doesn't mean we must expose it through getters/setters.

Why is this bad?

It can weaken **encapsulation**.

The main purpose of private members is:

```
Hide internal state
       ↓
Control access
       ↓
Maintain valid state
```

If we provide unrestricted setters:

```
user.setAge(-100);
```

we may allow invalid data.

Better Approach

Use getters/setters **only when needed**, with validation/control.

```
void setAge(int age) {
    if(age >= 0) {
        this->age = age;
    }
}
```

Remember

> **Not every private variable needs a getter/setter.**

🟩 8.Premature Optimization

Premature optimization means:

> **Optimizing the code too early, before knowing where the actual performance problem is.**

Your notes give the principle:

> **"Make it work, then make it fast."**

Correct approach

```
First:
Make it work
     ↓
Test correctness
     ↓
Measure performance
     ↓
Find bottleneck
     ↓
Optimize
```

Example

Suppose a brute-force solution works correctly:

```
Brute Force
     ↓
Correct Answer
```

First ensure correctness.

Then:

```
Measure
   ↓
Find slow part
   ↓
Optimize
```

Your notes specifically mention:

> **Brute Force Solution First, then optimize for performance.**

🟩 9.Overuse of Inheritance

Your notes mention that excessive inheritance:

- Leads to complex hierarchies
- Causes tight coupling

Example:

```
        Animal
          |
        Mammal
          |
```

```
          Dog
           │
       SpecialDog
           │
     SuperSpecialDog
```

This can become difficult to understand and maintain.

Alternatives

Your notes list:

```
Overuse of Inheritance
          │
          ├── Composition
          ├── Strategy Pattern
          ├── Visitor Pattern
          ├── Bridge Pattern
          └── Inheritance only for appropriate abstract relationships
```

Important

Inheritance should represent a genuine **is-a relationship**.

Composition often gives more flexibility:

```
Car
 ├── Engine
 ├── Transmission
 └── GPS
```

instead of creating a huge inheritance hierarchy.

🟩 Benefits of Avoiding Anti-Patterns

Your notes list four benefits:

1️⃣ Better Code Quality

Code becomes:

```
Cleaner
Readable
Maintainable
```

2️⃣ Loosely Coupled Code

Classes become less dependent on each other.

```
A → B
```

instead of:

```
A ↔ B ↔ C ↔ D ↔ E
```

3️⃣ Improved Design Pattern Application

We use patterns where they actually solve a problem instead of unnecessarily applying them.

4️⃣ Easier New Feature Integration

Clean architecture makes adding new functionality easier.

```
Existing System
      ↓
New Feature
      ↓
Minimal Changes
```

🟦 PART 2 — Null Object Pattern

🟩 What is Null Object Pattern?

Null Object is a design pattern used to **avoid repeated null checks**.

Normally we write:

```
if(obj != nullptr) {
    obj->method();
}
```

If this appears everywhere:

```
if(obj != null)
if(obj != null)
if(obj != null)
```

```
if(obj != null)
...
```

code becomes cluttered.

Null Object solution:

Instead of giving `null`, give a **special object that does nothing**.

```
Real Object
    OR
Null Object
```

Both implement the same interface.

🟩 Purpose of Null Object Pattern

Your notes list three purposes:

1️⃣ Avoid Null Checks

Instead of:

```
if(obj == null)
```

we provide a Null Object.

2️⃣ Prevent NullPointerException

Instead of:

```
null → method call → runtime error ❌
```

we have:

```
Null Object → method call → safe behavior ✅
```

3️⃣ Replace Conditionals with Polymorphism

Instead of:

```
if(obj != null)
    obj->m();
```

```
else
    // do nothing
```

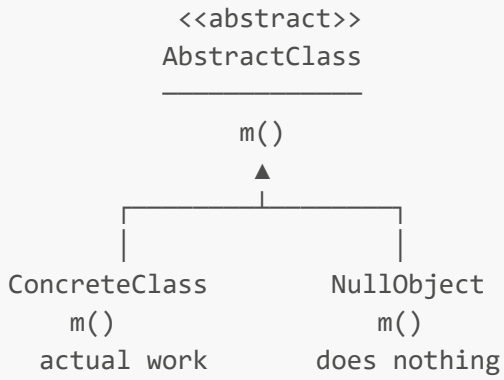
we can simply:

```
obj->m();
```

The actual object decides what happens.

🟩 Null Object Concept

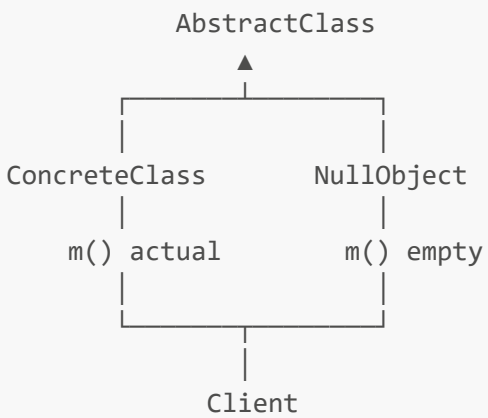
Suppose we have:



Now client doesn't need to know whether it received a real object or a Null Object.

🟩 Client Flow

Your diagram shows:



Client simply works with:

```
AbstractClass* obj;
```

It doesn't care whether:

```
obj = ConcreteClass
```

or:

```
obj = NullObject
```

🟩 How Null Object Works

Suppose normal code is:

```
void func(AbstractClass* obj) {

    if(obj != nullptr) {
        obj->m();
    }
}
```

With Null Object:

```
void func(AbstractClass* obj) {
    obj->m();
}
```

If we don't have a real object:

```
obj → NullObject
```

Then:

```
obj->m();
```

calls:

```
NullObject::m()
```

which does nothing or returns a default value.

🟩 Creating a Null Object

Your notes give these steps:

Step 1

Create a **Null Object class**.

Step 2

Make it inherit/implement the same abstract class/interface.

Step 3

Implement methods such that they:

```
Do nothing
       OR
Return default values
```

Examples:

```
int → 0

string → ""

bool → false
```

Step 4

Client always receives an object reference:

```
Concrete Object
       OR
Null Object
```

Therefore client doesn't need repeated null checking.

🟩 Why Null Object?

This is an important distinction from your page 6.

Null Reference

A null reference:

```
obj → nothing
```

It points to nothing in the heap.

If we do:

```
obj->method();
```

we can get a runtime error such as a **NullPointerException** in languages where such an exception exists.

Null Object

A Null Object is an **actual object**:

```
obj
 ↓
NullObject
```

It exists in memory but represents:

> "There is no meaningful real object to perform this operation."

Its methods safely do nothing or return default values.

Easy Difference

```
NULL
 ↓
Nothing ❌

NULL OBJECT
 ↓
Actual empty/do-nothing object ✅
```

🟩 Null Object and Liskov Substitution Principle

Your notes mention:

> **Avoids Liskov Substitution Principle break**

The idea is that the Null Object follows the same interface/contract as the normal object.

For example:

```
ICommand
   ▲
   ├── LightOnCommand
   └── NoCommand
```

Both can be used where an `ICommand` is expected.

The `NoCommand` simply performs no action.

So client doesn't need a special case.

🟩 Example 1 — Strategy Pattern

Your notes connect Null Object with the **Strategy Design Pattern**, using the Robot example.

Earlier we had:

```
Robot
  │
  └── IFlyStrategy
```

Some robots can fly:

```
Robot
  ↓
FlyStrategy
  ↓
fly()
```

But some robots cannot fly.

Instead of:

```
if(robotCanFly) {
    flyStrategy->fly();
}
```

we create:

```
NoFlyBehaviour
```

NoFlyBehaviour

```
class NoFlyBehaviour : public IFlyBehaviour {
public:
    void fly() {
        // do nothing
    }
};
```

So:

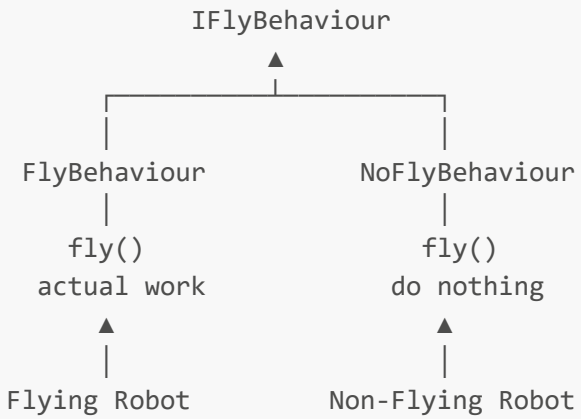
```
Flying Robot
      ↓
FlyBehaviour

Non-Flying Robot
      ↓
NoFlyBehaviour
```

Client can always call:

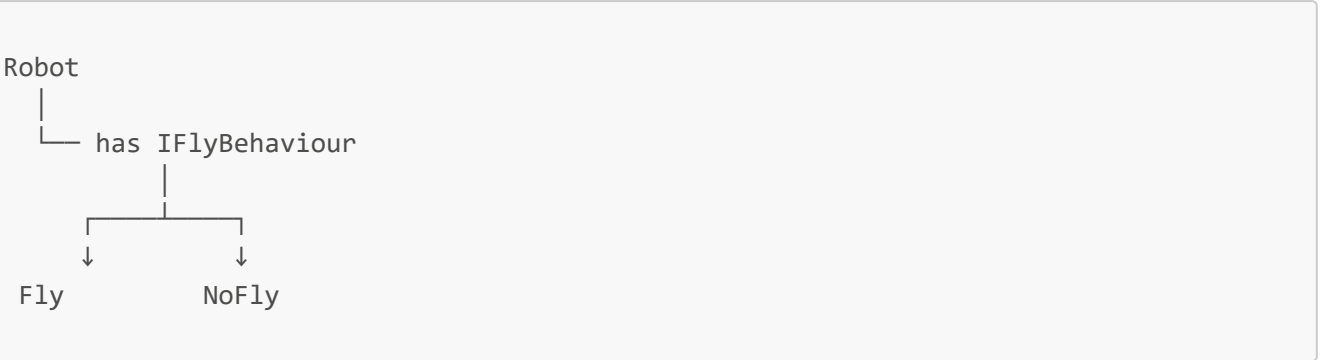
```
flyBehaviour->fly();
```

No null check is required.

🟩 Strategy + Null Object Diagram



Flow



This avoids:

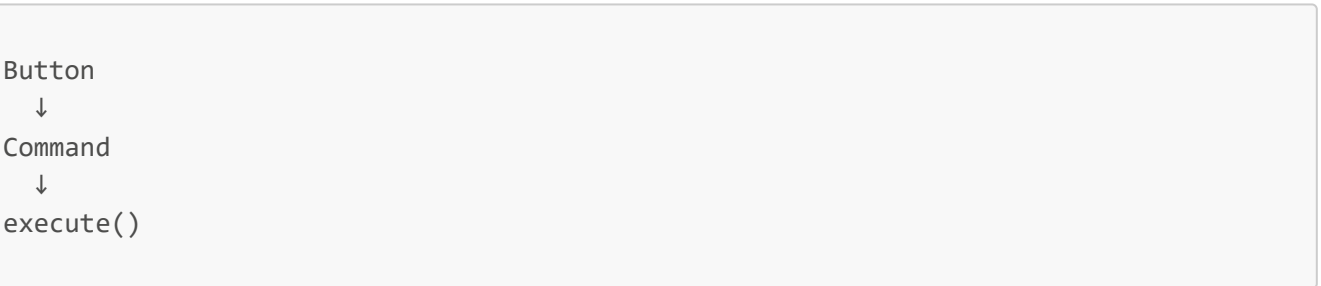
```
if(flyBehaviour != nullptr)
```

🟩 Example 2 — Command Design Pattern

Your notes also connect Null Object with the **Command Design Pattern**, using the Remote Control example.

Suppose remote control has a button.

Normally:



But what if no command is assigned?

Without Null Object:

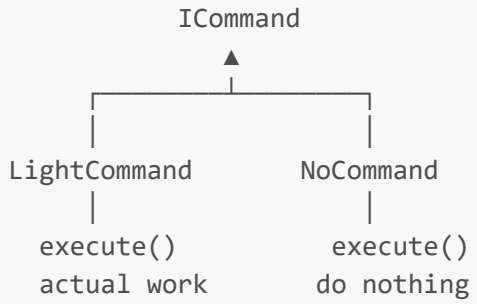
```
if(command != nullptr) {
    command->execute();
}
```

Null Object Solution

Create:

```
NoCommand
```

which implements the same Command interface.



Initialize the button with:

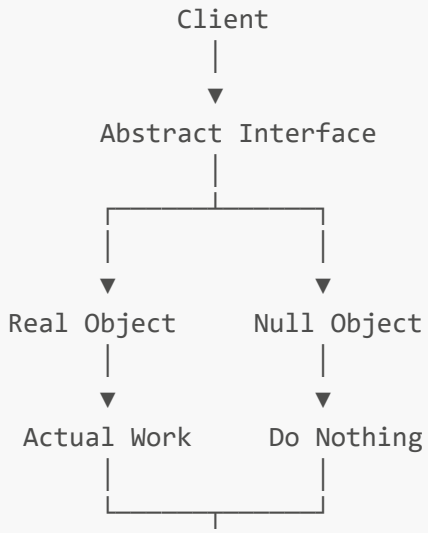
```
Button → NoCommand
```

Now whenever the button is pressed:

```
button.press()
      ↓
command.execute()
      ↓
NoCommand.execute()
      ↓
Nothing happens safely
```

No null check is required.

🟩 Complete Null Object Flow



```
                          ▼
                    Same Interface
```

Main benefit:

```
Before:

if(obj != null)
    obj->method();

After:

obj->method();
```

Because `obj` is **always an object**.